МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Звіт

про виконання лабораторної роботи №1

"Організація професійного Python-проєкту та Git"

Виконав:

Студент 3 курсу

групи ФеП-33

Абдулханов Абдул-Рахім

Викладач:

проф. Свелеба С. А.

Львів-2026

Мета роботи: Навчитися організовувати Python-проєкт із src-layout, створювати віртуальне середовище, пакет і модулі, налаштовувати pyproject.toml, працювати з Git та оформлювати README. Індивідуальний варіант 1: система обліку студентів.

Теоретичні відомості:

Python-проєкт об'єднує пакет із кодом, конфігурацію, документацію й перевірки. У src-layout вихідний код відокремлений від інших файлів. Середовище .venv ізолює встановлені пакети, pyproject.toml задає метадані й спосіб встановлення, а Git зберігає послідовність змін. Type hints описують очікувані типи, але не замінюють перевірку даних під час виконання.

Виконання роботи

1. Визначив завдання варіанта 1: створити систему обліку студентів. Для кожного запису зберігаю ім'я, прізвище, групу та середній бал. Реалізував додавання, перегляд, пошук за групою, найкращого студента й середній бал групи.

2. Створив ізольоване середовище .venv та перевірив інтерпретатор. Для роботи використав Python 3.12.14; пакет сумісний із Python 3.11 і новішими. Активація PowerShell показана командою з крапкою на початку.

```
PS F:\university\3yr-1\professional_python> uv run --python 3.12 --no-project python -m venv .venv; .
.\.venv\Scripts\Activate.ps1; python --version; python -m pip --version
Python 3.12.14
pip 25.0.1 from F:\university\3yr-1\professional_python\.venv\Lib\site-packages\pip (python 3.12)
PS C:\Users\ponti> _
```

3. Організував src-layout. Модуль models.py описує студента, services.py виконує операції з даними, main.py відповідає за консоль. __init__.py містить версію пакета. Тести й матеріали роботи лежать окремо. Нижче показано перелік відстежуваних файлів до додавання PDF.

```
PS F:\university\3yr-1\professional_python> git ls-files
.gitignore
README.md
lab1/task/Lab_01.docx
pyproject.toml
src/student_manager/__init__.py
src/student_manager/main.py
src/student_manager/models.py
src/student_manager/services.py
tests/__init__.py
tests/test_cli.py
tests/test_models.py
tests/test_services.py
PS C:\Users\ponti> _
```

4. Описав пакет у pyproject.toml: назву student-manager, версію 0.1.0, автора, Python >=3.11 та setuptools як backend. Залежностей для запуску немає. Секція

project.scripts пов’язує команду student-manager із функцією main(), а пошук пакетів починається з src/.

```
[build-system]
requires = ["setuptools>=70"]
build-backend = "setuptools.build_meta"

[project]
name = "student-manager"
version = "0.1.0"
description = "Student management system for Professional Python, variant 1"
readme = "README.md"
requires-python = ">=3.11"
authors = [{name = "Абдулханов Абдул-Рахім"}]
dependencies = []

[project.scripts]
student-manager = "student_manager.main:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.ruff]
target-version = "py311"
line-length = 88
```

У .gitignore виключив середовище, кеші, секрети та проміжні файли звіту. Для завдань у lab*/task/ залишив виняток, щоб оригінальний DOCX викладача був у репозиторії.

```
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
.mypy_cache/
.ruff_cache/
.idea/
.vscode/
dist/
build/
*.egg-info/
.env
*.docx
!lab*/task/*.docx
*.loop
*conspect*
guides/
plots/
temp_plots/
code_screenshots/
build_report.py
```

```
lab*/report_assets/
lab*/evidence/
```

Встановив пакет у режимі editable. Після цього зміни коду доступні без повторного встановлення. Нижче наведено повторну перевірку встановлення без службового журналу та список пакетів.

```
PS F:\university\3yr-1\professional_python> . .\.venv\Scripts\Activate.ps1; python -m pip install --quiet --disable-pip-version-check -e .; python -m pip list --disable-pip-version-check
Package         Version Editable project location
--------------- ------- ------------------------------------
pip             25.0.1
student-manager 0.1.0   F:\university\3yr-1\professional_python

PS C:\Users\ponti> _
```

5. Створив Student через dataclass. Під час створення прибираю пробіли на краях полів, не приймаю порожні ім’я, прізвище чи групу та перевіряю бал у межах 0-100. Властивість full_name об’єднує ім’я та прізвище.

```python
from dataclasses import dataclass


@dataclass
class Student:
    first_name: str
    last_name: str
    group: str
    average_grade: float

    def __post_init__(self) -> None:
        self.first_name = self.first_name.strip()
        self.last_name = self.last_name.strip()
        self.group = self.group.strip()
        if not self.first_name or not self.last_name or not self.group:
            raise ValueError("Name, surname and group cannot be empty")
        if not 0 <= self.average_grade <= 100:
            raise ValueError("Average grade must be between 0 and 100")

    @property
    def full_name(self) -> str:
        return f"{self.first_name} {self.last_name}"
```

6. Виніс бізнес-логіку в services.py. Додавання повертає створеного студента. Пошук групи не залежить від регістру; для порожнього набору найкращий студент і середнє повертають None. За однакового максимального бала вибирається перший запис.

```python
from student_manager.models import Student


def add_student(
    students: list[Student],
    first_name: str,
    last_name: str,
    group: str,
    average_grade: float,
) -> Student:
    student = Student(first_name, last_name, group, average_grade)
    students.append(student)
    return student


def filter_by_group(students: list[Student], group: str) -> list[Student]:
    group = group.strip().casefold()
    return [student for student in students if student.group.casefold() == group]


def find_best_student(students: list[Student]) -> Student | None:
    return max(students, key=lambda student: student.average_grade, default=None)


def calculate_group_average(students: list[Student], group: str) -> float | None:
    members = filter_by_group(students, group)
    if not members:
        return None
    return sum(student.average_grade for student in members) / len(members)
```

7. У main.py підготував чотири вигадані навчальні записи та окреме виведення списку. Демонстраційний режим показує всі потрібні операції без ручного введення.

```python
import argparse

from student_manager.models import Student
from student_manager.services import (
    add_student,
    calculate_group_average,
    filter_by_group,
    find_best_student,
)


def create_demo_students() -> list[Student]:
    return [
        Student("Олена", "Коваль", "ФеП-33", 91.5),
        Student("Іван", "Мельник", "ФеП-33", 84.0),
        Student("Ірина", "Бондар", "ФеП-32", 95.2),
        Student("Андрій", "Шевченко", "ФеП-33", 88.7),
    ]


def print_students(students: list[Student]) -> None:
    if not students:
        print("Студентів не знайдено.")
        return
    print(f"{'Ім'я та прізвище':<30} {'Група':<10} {'Середній бал':>12}")
    for student in students:
        print(
            f"{student.full_name:<30} {student.group:<10} "
            f"{student.average_grade:>12.2f}"
        )
```

```python
def run_demo() -> None:
    students = create_demo_students()
    print_students(students)
    print("\nСтуденти групи ФеП-33:")
    print_students(filter_by_group(students, "ФеП-33"))
    average = calculate_group_average(students, "ФеП-33")
    if average is not None:
        print(f"\nСередній бал групи ФеП-33: {average:.2f}")
    best = find_best_student(students)
    if best is not None:
        print(f"Найкращий студент: {best.full_name} ({best.average_grade:.2f})")
```

```
PS F:\university\3yr-1\professional_python> . .\.venv\Scripts\Activate.ps1; python -m student_manager.main
--demo
Ім’я та прізвище               Група      Середній бал
Олена Коваль                   ФеП-33            91.50
Іван Мельник                   ФеП-33            84.00
Ірина Бондар                   ФеП-32            95.20
Андрій Шевченко                ФеП-33            88.70

Студенти групи ФеП-33:
Ім’я та прізвище               Група      Середній бал
Олена Коваль                   ФеП-33            91.50
Іван Мельник                   ФеП-33            84.00
Андрій Шевченко                ФеП-33            88.70

Середній бал групи ФеП-33: 88.07
Найкращий студент: Ірина Бондар (95.20)
PS C:\Users\ponti> _
```

У групі ФеП-33 отримав 88.07: середнє від 91.5, 84.0 та 88.7. Найбільший бал у всьому наборі має Ірина Бондар: 95.20. Формат виведення округлює числа до двох знаків, а сам розрахунок зберігає float.

8. Додав введення з клавіатури й меню. Некоректний бал або порожнє поле не додає студента. Це разом із dataclass, валідацією та README з прикладом запуску покриває додаткові можливості із завдання.

```python
def read_student(students: list[Student]) -> None:
    first_name = input("Ім'я: ")
    last_name = input("Прізвище: ")
    group = input("Група: ")
    try:
        grade = float(input("Середній бал (0-100): "))
        add_student(students, first_name, last_name, group, grade)
    except ValueError:
        print("Помилка: заповніть усі поля та введіть числовий бал від 0 до 100.")
        return
    print("Студента додано.")
```

```python
def run_menu() -> None:
    students: list[Student] = []
    while True:
        print(
            "\n1. Додати студента\n2. Усі студенти\n3. Пошук за групою\n"
            "4. Найкращий студент\n5. Середній бал групи\n0. Вихід"
        )
        command = input("Команда: ").strip()
        if command == "0":
            return
        if command == "1":
            read_student(students)
        elif command == "2":
            print_students(students)
        elif command == "3":
            print_students(filter_by_group(students, input("Група: ")))
        elif command == "4":
            best = find_best_student(students)
            if best is None:
                print("Студентів не знайдено.")
            else:
                print(f"Найкращий студент: {best.full_name} ({best.average_grade:.2f})")
        elif command == "5":
            group = input("Група: ").strip()
            average = calculate_group_average(students, group)
            if average is None:
                print("Студентів не знайдено.")
            else:
                print(f"Середній бал групи {group}: {average:.2f}")
        else:
            print("Невідома команда.")

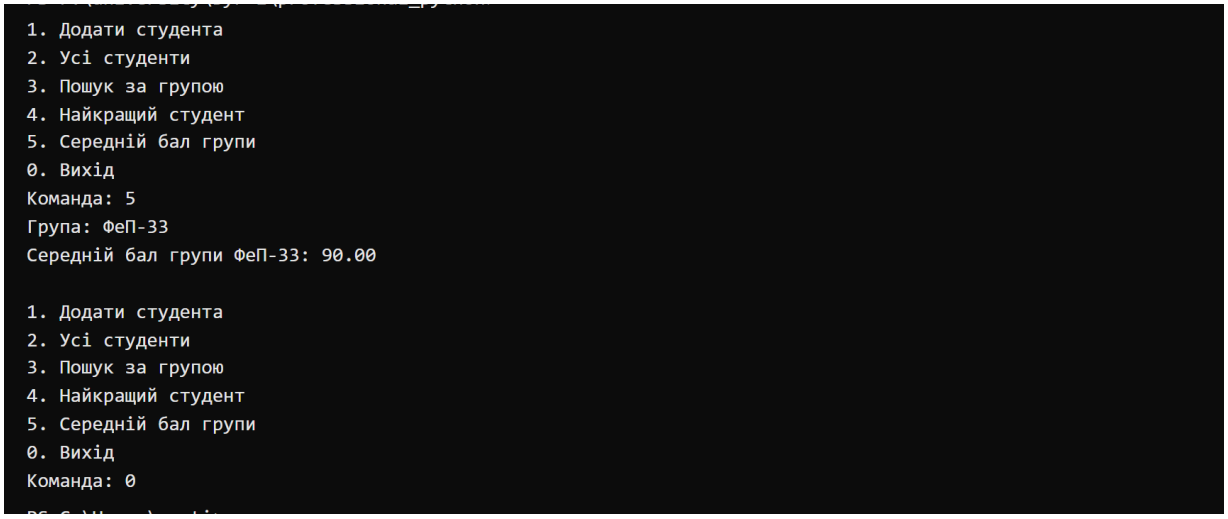

def main() -> None:
    parser = argparse.ArgumentParser(description="Система обліку студентів")
    parser.add_argument("--demo", action="store_true", help="показати приклад роботи")
    args = parser.parse_args()
    if args.demo:
        run_demo()
    else:
        try:
            run_menu()
        except (EOFError, KeyboardInterrupt):
            print()


if __name__ == "__main__":
    main()
```

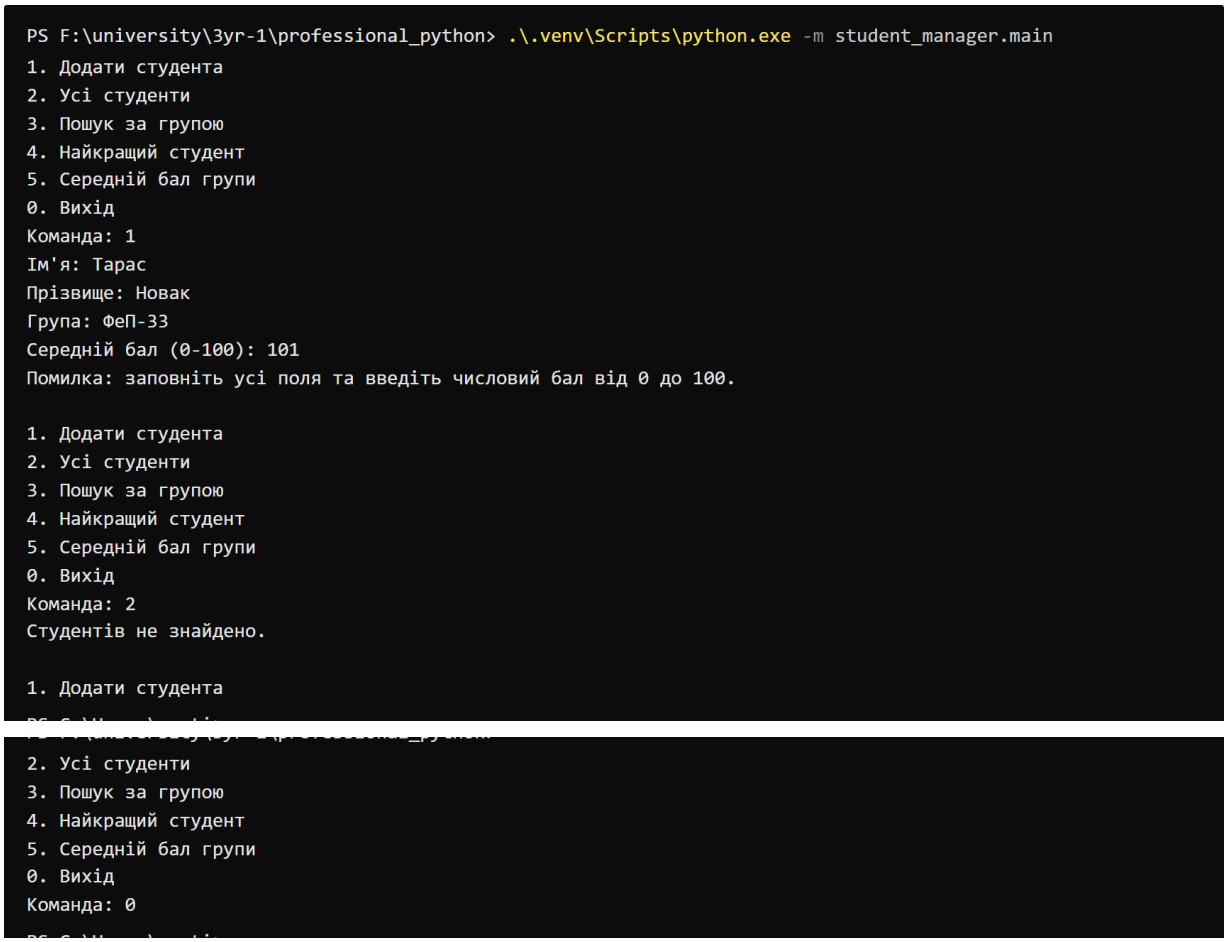
Перевірив введення Тараса Новака з балом 90, перегляд списку й середній бал групи. Довгу інтерактивну сесію нижче поділив на послідовні фрагменти.

```
PS F:\university\3yr-1\professional_python> .\.venv\Scripts\python.exe -m student_manager.main
1. Додати студента
2. Усі студенти
3. Пошук за групою
4. Найкращий студент
5. Середній бал групи
0. Вихід
Команда: 1
Ім'я: Тарас
Прізвище: Новак
Група: ФеП-33
Середній бал (0-100): 90
Студента додано.

1. Додати студента
2. Усі студенти
3. Пошук за групою
4. Найкращий студент
5. Середній бал групи
0. Вихід
Команда: 2
Ім’я та прізвище              Група      Середній бал
Тарас Новак                   ФеП-33            90.00
```



Окремо ввів 101. Програма повідомила про помилку, а список залишився порожнім.



9. Додав невеликі автоматичні перевірки стандартною бібліотекою unittest. Вони перевіряють поля, граничні й некоректні оцінки, пошук групи, порожні результати та роботу консольного меню. Нижче наведено приклад перевірки середнього бала й результат повного запуску тестів.

```python
    def test_group_average_uses_only_the_requested_group(self) -> None:
        services = importlib.import_module("student_manager.services")
        self.assertTrue(hasattr(services, "calculate_group_average"))
        students = [
            Student("Олена", "Коваль", "ФеП-33", 90.0),
            Student("Іван", "Мельник", "ФеП-33", 80.0),
            Student("Ірина", "Бондар", "ФеП-32", 100.0),
        ]
        self.assertAlmostEqual(
            services.calculate_group_average(students, " феп-33 "), 85.0
        )
        self.assertIsNone(services.calculate_group_average(students, "ФеП-99"))
        self.assertIsNone(services.calculate_group_average([], "ФеП-33"))
```

```
PS F:\university\3yr-1\professional_python> . .\.venv\Scripts\Activate.ps1; python -m unittest discover -s
tests -q
----------------------------------------------------------------------
Ran 13 tests in 1.467s

OK
PS C:\Users\ponti> _
```

10. Оформив README: зазначив автора й варіант, пояснив встановлення, запуск, перевірку та структуру. Фрагмент інструкції запуску:

````
## Запуск

Інтерактивне меню з порожнім списком студентів:

```powershell
python -m student_manager.main
```

Готовий приклад із чотирма навчальними записами:

```powershell
python -m student_manager.main --demo
```

Після встановлення доступна також команда `student-manager --demo`.
Меню: `1` - додати, `2` - список, `3` - пошук за групою,
`4` - найкращий студент, `5` - середній бал групи, `0` - вихід.
````

Перевірив також консольну команду, яку встановив pip. Вона запускає ту саму функцію main(), що й запуск модуля.

```
PS F:\university\3yr-1\professional_python> . .\.venv\Scripts\Activate.ps1; student-manager --demo
Ім’я та прізвище              Група      Середній бал
Олена Коваль                  ФеП-33            91.50
Іван Мельник                  ФеП-33            84.00
Ірина Бондар                  ФеП-32            95.20
Андрій Шевченко               ФеП-33            88.70

Студенти групи ФеП-33:
Ім’я та прізвище              Група      Середній бал
Олена Коваль                  ФеП-33            91.50
Іван Мельник                  ФеП-33            84.00
Андрій Шевченко               ФеП-33            88.70

Середній бал групи ФеП-33: 88.07
Найкращий студент: Ірина Бондар (95.20)
PS C:\Users\ponti> _
```

11. Зафіксував три логічні етапи в Git: структуру, модель із логікою та консоль із документацією. Відправив їх у публічний репозиторій. Нижче показано стан до додавання самого звіту; PDF буде окремим комітом.

```
PS F:\university\3yr-1\professional_python> git status; git log --oneline; git remote -v
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
279b365 Add interactive console menu and document Lab 1 usage
46dc7ca Add validated student model and student-management operations
5dbb045 Create installable student-manager project structure
origin  https://github.com/RakhimAbdulkhanov/professional_python.git (fetch)
origin  https://github.com/RakhimAbdulkhanov/professional_python.git (push)
PS C:\Users\ponti> _
```

Репозиторій: https://github.com/RakhimAbdulkhanov/professional_python

Обмеження першої роботи: дані зберігаються лише в пам’яті до виходу. Імпорт та збереження у файлах і БД додаватиму в наступних роботах.

Контрольні запитання

1. Що таке Python-проєкт?

Це код, конфігурація, залежності, документація й перевірки, об’єднані для розв’язання однієї задачі. У цій роботі це система обліку студентів.

2. Чим проєкт відрізняється від одного Python script?

Один скрипт може містити всю програму в одному файлі. Проєкт розділяє її на пакет і модулі, має правила встановлення та історію змін.

3. Що таке Python module?

Модуль - файл Python, який можна імпортувати. Наприклад, services.py містить функції додавання студентів і розрахунків.

4. Що таке Python package?

Пакет об’єднує модулі в одному просторі імен. Тут student_manager є пакетом із модулями models, services і main.

5. Для чого потрібний __init__.py?

Він позначає звичайний пакет і може містити його початкові налаштування або публічні імпорти. У моєму пакеті зберігається лише __version__.

6. Що таке src-layout?

Це структура, у якій код пакета лежить у каталозі src/, а не поряд зі звітами й конфігурацією в корені проєкту.

7. Які переваги має src-layout?

Він відділяє код від допоміжних файлів. Після встановлення перевіряється саме пакет, а не випадково доступні модулі з кореня репозиторію.

8. Для чого використовується virtual environment?

Ізольоване середовище зберігає залежності конкретного проєкту окремо від інших програм і системного Python.

9. Як створити .venv?

Командою python -m venv .venv. Потрібна версія інтерпретатора має бути вибрана перед виконанням команди.

10. Як активувати virtual environment?

У PowerShell: .\.venv\Scripts\Activate.ps1. У bash на Linux/macOS: source .venv/bin/activate; після активації python і pip беруться із середовища.

11. Для чого використовується pip?

pip встановлює та видаляє пакети Python і показує встановлені версії. У роботі він встановив локальний student-manager.

12. Чому рекомендується python -m pip?

Так pip запускається через конкретний Python. Це допомагає не встановити пакет в інше середовище через випадковий pip у PATH.

13. Для чого потрібний pyproject.toml?

Він описує метадані пакета, спосіб збирання та налаштування інструментів. У мене там також визначено консольну команду student-manager.

14. Що містить секція [project]?

Назву, версію, опис, автора, README, підтримувані версії Python і залежності. Підсекція [project.scripts] задає консольні точки входу.

15. Що таке editable installation?

Це встановлення через python -m pip install -e ., коли редагування вихідних .py файлів одразу змінює поведінку пакета. Для змін метаданих або точки входу встановлення може знадобитися повторити.

16. Для чого використовується if __name__ == "__main__"?

Ця перевірка запускає main() під час виконання модуля як програми. Звичайний імпорт main не повинен відкривати меню чи запитувати введення.

17. Що таке Git?

Git - розподілена система контролю версій. Вона зберігає історію змін і дозволяє порівнювати та відновлювати попередні стани проєкту.

18. Що таке repository?

Репозиторій містить файли проєкту та історію Git. GitHub зберігає віддалену копію, але сам Git працює й локально.

19. Що таке commit?

Коміт - зафіксований стан підготовлених змін із повідомленням, автором та ідентифікатором. Я робив окремі коміти структури, логіки й консольного інтерфейсу.

20. Для чого використовується git status?

Команда показує поточну гілку та стан файлів: підготовлені, змінені й невідстежувані. Чистий стан означає відсутність незакомічених змін у відстежуваних файлах.

21. Для чого git add?

git add переносить вибрані зміни в індекс, тобто готує їх до наступного коміту. Він сам не створює коміт і не надсилає файли на GitHub.

22. Для чого git commit?

git commit зберігає підготовлені зміни в локальній історії. Для відправлення на GitHub окремо виконується git push.

23. Що показує git log?

Історію комітів. Варіант git log --oneline показує скорочений ідентифікатор і повідомлення кожного коміту.

24. Для чого потрібен .gitignore?

Він задає шаблони невідстежуваних файлів, які Git має ігнорувати. На вже відстежувані файли ці правила автоматично не поширюються.

25. Чому .venv не потрібно комітити?

Середовище містить локальні виконувані файли й пакети та залежить від комп’ютера. Його відтворюють зі специфікації, а не передають у репозиторії.

26. Для чого потрібний README?

README пояснює призначення проєкту, встановлення, запуск і перевірку. Тут він також описує меню, структуру та обмеження зберігання даних.

27. Що таке type hints?

Це анотації очікуваних типів параметрів, результатів і змінних. Вони допомагають читати код та перевіряти його інструментами, але самі не виконують runtime-валідацію.

28. Для чого програму розділяють на modules?

Щоб розділити відповідальність і простіше змінювати та перевіряти код. Наприклад, розрахунок середнього бала не залежить від способу введення.

29. Що таке business logic?

Це правила предметної області. У моєму проєкті до них належать додавання студента, фільтрація групи та обчислення її середнього бала.

30. Чому потрібно відокремлювати application entry point від business logic?

Точка входу обробляє запуск та взаємодію з користувачем, а логіка виконує розрахунки. Завдяки цьому ті самі функції можна перевіряти тестами й пізніше викликати з API.

Висновок:

Створив пакет student_manager зі структурою src-layout і реалізував усі операції варіанта 1. Навчився відокремлювати модель, обчислення та консольний інтерфейс, встановлювати локальний пакет і вести історію Git. Перевірив роботу на навчальних даних, некоректному введенні та автоматичними тестами. Отримана структура є основою для наступних лабораторних робіт.